

RESEARCH

Open Access



# ProcSAGE: an efficient host threat detection method based on graph representation learning

Boyuan Xu<sup>1,2</sup>, Yiru Gong<sup>1,2</sup>, Xiaoyu Geng<sup>1,2</sup>, Yun Li<sup>1,2</sup>, Cong Dong<sup>3</sup>, Song Liu<sup>1,2\*</sup> , Yuling Liu<sup>1,2</sup>, Bo Jiang<sup>1,2</sup> and Zhigang Lu<sup>1,2</sup>

## Abstract

Advanced Persistent Threats (APTs) achieves internal networks penetration through multiple methods, making it difficult to detect attack clues solely through boundary defense measures. To address this challenge, some research has proposed threat detection methods based on provenance graphs, which leverage entity relationships such as processes, files, and sockets found in host audit logs. However, these methods are generally inefficient, especially when faced with massive audit logs and the computational resource-intensive nature of graph algorithms. Effectively and economically extracting APT attack clues from massive system audit logs remains a significant challenge. To tackle this problem, this paper introduces the ProcSAGE method, which detects threats based on abnormal behavior patterns, offering high accuracy, low cost, and independence from expert knowledge. ProcSAGE focuses on processes or threads in host audit logs during the graph construction phase to effectively control the scale of provenance graphs and reduce performance overhead. Additionally, in the feature extraction phase, ProcSAGE considers information about the processes or threads themselves and their neighboring nodes to accurately characterize them and enhance model accuracy. In order to verify the effectiveness of the ProcSAGE method, this study conducted a comprehensive evaluation on the StreamSpot dataset. The experimental results show that the ProcSAGE method can significantly reduce the time and memory consumption in the threat detection process while improving the accuracy, and the optimization effect becomes more significant as the data size expands.

**Keywords** Graph representation learning, Host threat detection, Provenance graph

## Introduction

In the context of rapidly improving efficiency in vulnerability exploration and the proliferation of malicious software, both government entities and enterprises are placing considerable emphasis on cyber security (Binde et al. 2011). Different from conventional attacks, APT attacks are characterized by a long time scale and well chain. In an APT scenario, adversaries typically conduct

comprehensive reconnaissance to identify deployed software and corresponding versions, then exploit zero-day vulnerabilities to bypass rule-based host security strategies. Once these vulnerabilities are exploited, adversaries attempt to escalate privileges, obfuscate their activities, and maintain a low profile for an extended duration, before executing actions such as system destruction or exfiltration of sensitive data, in accordance with the adversary's directives (Alshamrani et al. 2019; Zimba et al. 2020). To counter APT attacks, the U.S. Department of Defense initiated the DARPA Transparent Computing Program in 2015. This program aims to research high-fidelity and visualization methods capable of uncovering malicious activities within systems (Han et al. 2020).

In an APT attack, adversaries typically implant malware or exploit active service components on servers to

\*Correspondence:

Song Liu

liusong1106@iie.ac.cn

<sup>1</sup> Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China

<sup>2</sup> School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

<sup>3</sup> Zhongguancun Laboratory, Beijing, China

gain prolonged and covert control of hosts. As defenders, implementing process-level monitoring mechanisms for each individual process can be complex and inefficient. Therefore, employing system audit logs at the system kernel level to observe and audit all processes is recognized as a viable solution.

System audit logs serve as comprehensive records of system activities and events. It records every syscall made by all processes (or threads) during their execution in the system. Whether it's a malicious process authored by an attacker or a service component process maliciously exploited by a victim, they all leave traces in the system call audit log. Security managers rely on these logs to ensure system security, as well as to monitor and troubleshoot system issues effectively. Typical audit logs encompass a wide range of information, including user login events, access to system resources, and various system events such as system startup, shutdown, hardware failures, and network events, among others. In implementation, the system audit log can be acquired using kernel-level frameworks like Auditd and Camflow (Pasquier et al. 2017).

However, the problem brought by the comprehensive information of the system audit log is the explosive growth of volume. Only one Linux system can generate gigabytes of audit data in a single day. This problem is worse especially when dealing with long-term APT attacks. Security administrators can hardly mine penetration clues from these massive data. Therefore, it is required to propose intelligent detection methods to identify penetration activities. Previous methods attempted to describe host behavior by constructing sequences of events and then determining whether a host was under attack. However, these sequence-based methods struggle to correlate events over long time scales. In recent years, with the development of graph convolutional neural network algorithms, researchers have proposed detection methods based on provenance graphs.

Provenance graphs organize system call audit logs in the form of graphs, which is more intuitive to reflect the information flow. This method first extracts entities (mainly processes, threads, files, and sockets) from the original logs. Then it builds a graph based on their syscall relationships and effectively associates events over extended time spans. In application, the provenance graph is not only used for detecting anomalous system activity on hosts. Besides, it is also used for tracing evidence, reconstructing attack scenarios, and supporting alert verification and investigation.

Although several studies have applied provenance graphs, they are mainly facing the efficiency challenge. First, as aforementioned, the source of the system audit logs leads to an explosive increase in the size of the

provenance graph. Second, the time and space complexity of graph algorithms is sensitive to graph size, resulting in memory explosions and exponential time growth. To solve the efficiency problem, previous research proposed various optimization methods, such as provenance graph compression, optimized processing schemes, and so on. They mainly focused on graph compression algorithms or graph representation algorithms after constructing the complete provenance graph. Hence, the optimization effectiveness is limited. Instead, we propose the ProcSAGE method which optimizes the efficiency of the provenance graph construction procedure. It considers the characteristics of provenance graphs with the needs of host anomaly detection. In summary, ProcSAGE performs advantages in high accuracy and efficiency, especially in low complexity.

The main contributions of this paper can be summarized as follows:

1. The paper proposes a new host anomaly detection method based on the provenance graph that does not rely on expert knowledge and effectively detects unknown attacks based on anomalous behavior patterns.
2. Instead of the inefficiency using of the entire system audit logs, the ProcSAGE method focuses only on the operations of processes (or threads) based on provenance graph xxx characteristics, which can significantly reduce the performance overhead.
3. Based on the ProcSAGE method's requirement for extracting features of processes (or threads), the paper proposed an effective feature extraction method. This method collects information from the processes (or threads) themselves and neighboring nodes in the provenance graph, providing a more accurate characterization of processes (or threads) and improving model accuracy.
4. Through benchmark on the StreamSpot dataset, experimental results indicate that ProcSAGE reduces time consumption by 69% and memory consumption by 78%, while maintaining performance comparable to other models.

## Related works

The work in this paper involves the host anomaly detection models based on system call audit logs, which can be categorized chronologically as traditional sequence-based methods, rule-based provenance graph methods, and anomaly-based provenance graph methods. Simultaneously, when utilizing provenance graphs, it is essential to consider the computational cost associated with graph algorithms. Therefore, in this section, we also survey the

relevant research on the performance optimization of these detection methods.

### Traditional sequence-based methods

In the early stages of host anomaly detection research, researchers tend to use event sequences to describe hosts' behavior, a practice referred to as traditional detection methods. For instance, in the work of Forrest et al. (1996), a sliding window of small length  $k$  (e.g.,  $k = 6$  Tan and Maxion 2002) is used to define the behavior patterns of each process based on syscalls. If a process deviates from the observed syscall sequence set during the initial analysis, it is considered anomalous, triggering an alert. However, this kind of method that inspects the syscall sequences of processes has its shortcomings. Wagner and Soto (2002) point out in the context of mimicry attacks that, as this host anomaly detection scheme relies on the normality statistics of syscalls, attackers can model their actions in order to distinguish them from benign processes. For example, a malicious process can insert meaningless or irrelevant syscalls to bypass the detection scheme of Han et al. (2020). Subsequent works have also highlighted this vulnerability (Forrest et al. 1996; Kruegel et al. 2005; Tan and Maxion 2002; Warrender et al. 1999). To address the weaknesses of the traditional sequence-based methods, researchers in recent years have started exploring host anomaly detection based on provenance graphs. In contrast to sequence-based methods, provenance graphs have the capability to link audit log entries that are temporally distant but share the same calling process or entity. This proves highly effective in detecting APT attacks with extended dwell times and slow-paced tactics. Furthermore, provenance graph methods not only consider the temporal relationships of audit log events but also delve more into causality relationships, which can significantly enhance the effectiveness of host anomaly detection.

### Rule-based provenance graph methods

Early works on provenance graphs almost employ rule-based methods to detect abnormal processes within a host, which are characterized by their simplicity, efficiency, and high accuracy. However, in practical applications, rule-based methods often heavily rely on expert knowledge and may exhibit poor transferability to different scenarios. SLEUTH (Hossain et al. 2017) assigns trustworthiness tags (t-tags) and confidentiality tags (c-tags) to entities such as processes and files. It then employs a set of inference rules to automatically label new nodes based on the labels of existing nodes. Alarms are generated if a subgraph matching predefined triggering alert rules appears in the provenance graph. While this method is operationally straightforward, its model

performance is highly dependent on the initial labeling of nodes. Holmes (Milajerdi et al. 2019a), on the other hand, constructs threat subgraphs as a knowledge graph created by security experts. It uses a hierarchical strategy template to map lower-level entity behaviors to Tactics, Techniques, and Procedures (TTPs) in the ATT &CK matrix. The model defines 16 types of TTP rules across seven stages of APT attacks and leverages graph-matching algorithms to identify matching attacks in the system provenance graph, achieving semantic threat detection and distinguishing the stage of the attack. This model maps the lower-level provenance graph to a higher conceptual layer, providing good interpretability. However, the mapping rules are fixed, leading to lower flexibility and scalability. POIROT (Milajerdi et al. 2019b), on the other hand, utilizes threat intelligence on known APT attacks to manually construct threat query graphs. It detects threats by aligning these graphs with the system provenance graph. This model performs well when faced with known attacks, even if the attackers attempt to conceal the attack chain. However, its efficiency is challenging to improve, and it cannot handle unknown attacks effectively. ProvG-Searcher (Altinisik et al. 2023), similar to POIROT, formulates the host anomaly detection problem as a subgraph matching task on provenance graphs. ProvG-Searcher embeds subgraphs in a vector space to directly evaluate subgraph relationships. By using order embeddings to streamline subgraph matching, ProvG-Searcher increases processing speed. However, it remains limited to the detection of known attacks.

### Anomaly-based provenance graph methods

Anomaly-based provenance graph detection methods, in contrast to rule-based approaches, place greater emphasis on the structural advantages of the provenance graph itself, leveraging contextual associations within the provenance graph to discover attacks. ZePro (Sun et al. 2018) follows a probabilistic approach to identify zero-day attack paths by inferring unknown attacks based on known ones. The authors build a Bayesian network using the provenance graph. Through statistical analysis of nodes that have already been identified as targets of attacks, the Bayesian network assesses the probability of infection for the remaining nodes. Objects with high infection probabilities are likely to have vulnerabilities, making them potential zero-day attack nodes. This model is effective in detecting unknown attacks but relies on expert knowledge for the discovery of known attacks. To reduce such dependence, ProvDetector (Wang et al. 2020) uses a rarity-based path selection algorithm to identify causal paths in the provenance graph representing potential malicious behavior of processes. It then employs a doc2vec embedding model and an outlier

detection model to determine the maliciousness of these paths, achieving hidden malicious process detection. StreamSpot (Manzoor et al. 2016a) considers the frequency of different patterns in the provenance graph, proposing a shingling-based timestamped graph similarity function to extract summaries from heterogeneous ordered graphs. It designs a streamhash to maintain these summaries and uses an online clustering and anomaly detection scheme based on centroids to discover anomalies. A similar approach is also employed in Unicorn (Han et al. 2020), where the model first utilizes the Weisfeiler-Lehman algorithm to gather information about nodes and their neighboring nodes, creating a pattern histogram. It then extracts features from the histogram using HistoSketch and applies the K-medoids algorithm to cluster features, thereby detecting anomalies. threaTrace (Wang et al. 2022) adopts a multi-model framework based on GraphSAGE, detecting abnormal nodes by predicting node types and assessing deviations from actual types, enabling the detection of smaller-than-average abnormal nodes in covert intrusion activities in the provenance graph. KAIROS (Cheng et al. 2023) utilizes a graph neural network-based encoder-decoder architecture to learn the temporal evolution of structural changes in a provenance graph, allowing KAIROS to quantify the degree of anomaly for each system event and reconstruct attack footprints. While these anomaly-based detection methods provide excellent accuracy and scalability for real-time detection, they run into performance issues when dealing with large provenance graphs.

### Optimization methods for provenance graph

Using provenance graphs for host anomaly detection yields significantly better results than other methods. However, as log volumes increase, the consumption of time and space generated by the graph will be hard to accept. Therefore, researching how to optimize the performance of provenance graph methods is a valuable direction. One intuitive optimization strategy is to reduce. LogGC (Lee et al. 2013) attempts to reduce the number of nodes in the provenance graph. Inspired by the garbage collection concept in languages like Java, LogGC proposes a method to reduce entity nodes based on their lifecycle, deleting temporary file nodes that are no longer used after a process ends. Xu et al. (2016) try to reduce the edges of the provenance graph. Starting from the equivalence of causal relationships in the logs, they propose causality-preserving reduction, process-centric causality approximation reduction, and reduction based on domain knowledge. They delete redundant read/write operations unrelated to the target file while preserving causality relationships and using domain knowledge for reduction. Zhu et al. (2021) compress the entire graph

based on global semantics and suspicious semantics. They ensure global semantics by removing redundant events that do not affect global dependencies and focus on high-value files and sensitive process command lines to ensure that suspicious semantics related to attacks are not deleted. DepComm (Xu et al. 2022) partitions a provenance graph into process-centric communities using pre-defined random walk schemes and extracts for each community the paths that illustrate the flow of information through it. ProvG-Searcher (Altinisik et al. 2023) introduces a graph partitioning scheme and a behavior-preserving graph reduction method.

Another optimization approach is from the perspective of processing the provenance graph. Priotracker (Liu et al. 2018) and Nodoze (Hassan et al. 2019) evaluate based on the frequency of edges or paths. They analyze only the abnormal edges or paths, reducing the size of the original graph by two orders of magnitude. This ensures that important information about attacks is not lost and speeds up the investigation process. However, their methods require representative training data, and the quality of training data has a significant impact on compression efficiency. ThreaTrace (Wang et al. 2022) attempts to propose a data storage management scheme, saving most of the provenance graph data on disk and only taking a local graph into memory for processing.

Based on the observation and analysis of the data, this paper finds that most nodes in the provenance graph are files or sockets, and process or thread nodes constitute only a small part. Since all events during system operation are related to processes (or threads), this paper reduces the computation of the provenance graph from the entire graph to process nodes. This significantly reduces the time and memory consumption of the host anomaly detection algorithm while ensuring accuracy.

### Threat model

The threat model considered by the ProcSAGE method includes two types of attackers: external attackers who may launch Advanced Persistent Threat (APT) attack, and internal employees with limited host access privileges.

For external attackers, there are two potential stages of attack. First, attackers may probe the host's exposed interfaces to establish a foothold within the system. In this scenario, attackers could exploit the host's external services using methods such as process injection or exploiting vulnerabilities in these services to implant and execute malicious scripts. Second, attackers may have already gained access to the host system, remain hidden for an extended period of time, and prepare to disrupt or sabotage normal business processes within the host or steal critical data.



Internal attackers can more easily gain host access privileges through legitimate means compared to external attackers. They may be capable of executing malicious scripts, disrupting normal business processes, and stealing sensitive data.

In summary, the ProcSAGE method assumes that the attacker's behavior includes at least one of the following: creating new malicious processes or interfering with related processes or threads, resulting in inconsistencies with normal behavior.

Concerning the operating environment, the ProcSAGE method assumes the following conditions: First, the integrity of the operating system, log collection tools, and analysis tools should be ensured during installation and throughout the intrusion period without unauthorized modification or tampering. Second, the log information should be collected in its entirety and cannot be tampered. These requirements can be met by transmitting the collected logs to a log server in real-time, thus ensuring the integrity and trustworthiness of the log information.

For provenance graph threat detection methods such as ProcSAGE that rely on anomaly detection, it is further assumed that attackers will not launch attacks before or during model execution and training. In other words, it is assumed that the training data for the model comes from an environment that is not affected by attacks. Besides, during model training, attackers will not disrupt normal system operations or interfere with model training.

### ProcSAGE method

The ProcSAGE method detects APTs through anomaly detection based on provenance graph substructures. The novelty of ProcSAGE lies in its approach to focus graph representation learning on process (or thread) nodes rather than the entire provenance graph. This section provides a brief overview of the model's design, and then presents a detailed description of its specific implementation.

The proposition of the ProcSAGE method is based on the following two observations. First, attacks are always reflected in the processes, as adversaries often initiate new processes or interfere with existing ones in order to execute their attacks. Focusing monitoring on processes proves effective in detecting advanced persistent threats. Second, the number of processes is much smaller than the number of files (configuration files, data files, executable files, temporary files, etc.) and sockets involved in host operation, so the proportion of process (and thread) nodes in the provenance graph is relatively small. In the process of anomaly detection, focusing the monitoring on processes significantly reduces the computational

resource consumption and improves the detection efficiency.

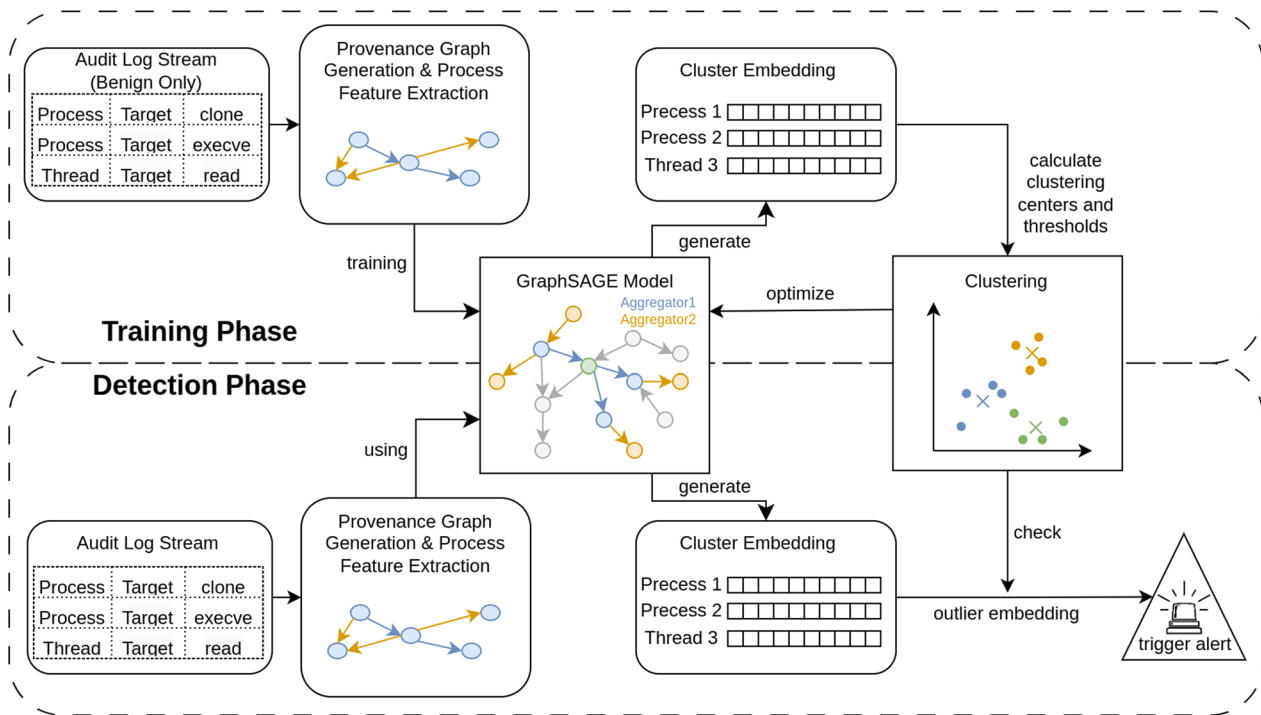
Similar to anomaly-based detection methods such as StreamSpot (Manzoor et al. 2016a), ProvDetector (Wang et al. 2020), and Unicorn (Han et al. 2020), ProcSAGE first characterizes the host behavior during the training phase and then continuously monitors the host behavior during the detection phase. Figure 1 illustrates the overall architecture of ProcSAGE. During the training phase, the ProcSAGE method constructs the provenance graph from the incoming audit log stream. The process semantic features and topological features are then extracted and concatenated as process features for all processes in the graph. The process features are subsequently used to train the GraphSAGE model to compute the cluster embedding for each process, ensuring the same labeled processes are in close proximity in Euclidean space. Lastly, the clustering centers and thresholds are calculated, which are used to define whether the process is anomalous or not. During the detection phase, the ProcSAGE method builds a provenance graph based on the audit log stream and extracts the features of each process using the same approach. The trained GraphSAGE model is then utilized to compute the clustering embedding vectors of the processes. The embedding vectors are then compared to the previously computed clustering centers and thresholds. An alert is triggered if the thresholds are exceeded.

### Provenance graph construction

To capture the complete attacker trace in APTs, the ProcSAGE method captures all lower-level audit log event streams including all syscalls and relevant entities to construct a provenance graph.

First of all, we try to provide the definition of the provenance graph used in this paper. In general, a provenance graph can be represented as  $G = \langle \text{Subject}, \text{Object}, \text{Event} \rangle$ . The *subject* represents the process or thread that initiates the syscall; the *object* typically refers to files, pipes, or sockets; *Event* represents a collection of syscalls, such as *read*, *write*, *fork*, *open*, etc., that indicate the associations between entities (*subjects* or *objects*). In the provenance graph, *subjects* and *objects* are represented as nodes, while *event* are represented as edges, allowing for multiple edges between two nodes.

The ProcSAGE method constructs a provenance graph by generating a directed edge for each syscall record in the system audit log stream. The direction of the edge depends on the flow of information in the corresponding syscalls. For syscalls such as *recv*, *exec*, and *read*, information flows from the object to the subject, creating an



**Fig. 1** ProcSAGE Method Overview

edge direction from the object to the subject. Conversely, for syscalls such as *clone*, *open*, *unlink*, *send*, and *write*, information flows from the subject to the object, creating an edge direction from the subject to the object.

In contrast to simply directing the edge from the *subject* to the *object*, this construction method better preserves the flow information on the provenance graph. As only processes or threads in the host can initiate syscalls, the direction of some subject-object edge is predictable (for example, the edge between a process node and a file node can only go from the process node to the file node). At this point, using directed edges from subject to object has the same information content as using undirected edges. However, determining the direction of the directed edge based on the flow of information allows more information to be retained in the directed graph.

After constructing the provenance graph, the ProcSAGE method extracts subgraphs from it for later feature extraction. For the provenance graph is heterogeneous, different types of edges (syscalls) coexist in the graph. In this case, the relationship between adjacent nodes may not have good interpretability. To overcome this problem, the ProcSAGE method proposes to select syscalls of the same type to form syscall groups, and then extract subgraphs corresponding to these syscalls. For instance, choose syscalls that involve process management to form a syscall group, such as *fork*, *clone*, *waitpid*, etc., to form

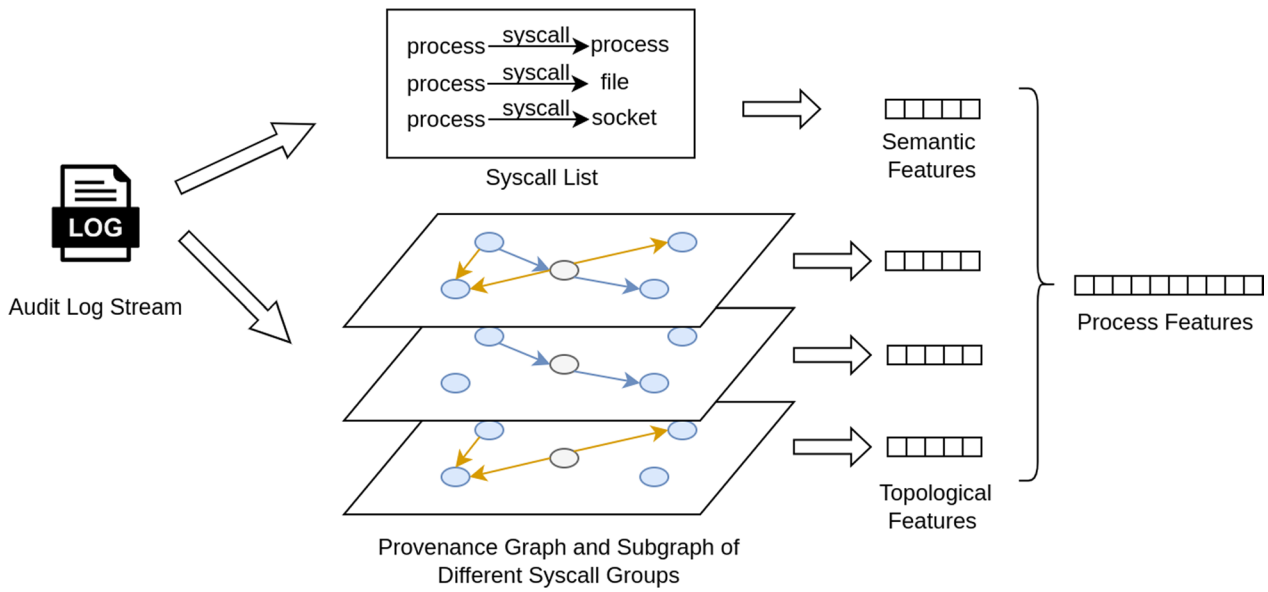
a subgraph. This subgraph will effectively reflect the features of different types of processes in the master-worker architecture. The Master process will have a greater number of neighbors, while the Worker node will have fewer neighbors.

### Feature extraction

The core concept of ProcSAGE involves process-oriented learning of graph representations. In order to accurately characterize process descriptions, it is essential to extract distinctive features from processes. As shown in Fig. 2, The selected process features in this paper consist of two components: semantic features and topological features. The topological features are derived from two graph algorithms applied to the provenance graph(or to its subgraph): the average degree of neighboring nodes  $k_{avg}(v)$  and the local clustering coefficient  $C(v)$ .

*The semantic features* of a process include information inherent to the process itself, such as the process name and the frequency of each type of syscall within the defined window time. These features provide direct insight into the characteristics and behavioral patterns of the process.

*The topological features* of a process refer to the characteristics of a process node with other neighboring nodes in the provenance graph (or in its subgraph). These topological features can portray the interaction



**Fig. 2** Feature Extraction in ProcSAGE

characteristics of the process with other nodes during its operation, including the relationship between the process and other processes or files, the frequency of interaction, and the mode of interaction.

The *average degree of neighboring nodes* refers to the average number of connections that a node's immediate neighbors have. For a directed provenance graph  $G$ , let  $N(v)$  be the set of neighbors representing the neighbors of a node  $v$ . This set can be further refined into the sets of in-neighbors  $N_{in}(v)$ , out-neighbors  $N_{out}(v)$ , and all neighbors  $N_{all}(v)$ . Additionally,  $d(v)$  represents the degree of a node  $v$ . The average degree of the neighbors  $k_{avg}(v)$  can be expressed as:

$$k_{avg}(v) = \frac{1}{|N(v)|} \sum_{u \in N(v)} d(u)$$

Clearly, the interpretability of the average degree of neighboring nodes is lacking in the overall provenance graph computation. Therefore, it is advisable to extract the average degree of neighbors on the syscall group subgraph, in order to present the neighborhood characteristics of process nodes across different subgraphs.

The *local clustering coefficient* measures the extent to which nodes in a network tend to cluster together, which can be expressed as:

$$C(v) = \frac{2 \times T(v)}{d(v) \times (d(v) - 1)}$$

where  $T(v)$  represents the number of triangles connected to node  $v$ , defined as:

$$T(v) = \frac{1}{2} \sum_{u, w \in N(v)} I_{\{\{u, w\} \in E(G)\}}$$

where  $I_{\{\{u, w\} \in E(G)\}}$  is the indicator function, which is 1 when  $\{u, w\}$  is an edge in graph  $G$ , and 0 otherwise.

From the formula, it's clear that when calculating the local clustering coefficient of a node, the focus is on the triangles with that node as a vertex. Triangles are quite common in general graph structures. However, in the provenance graph, due to the fact that there are only "process-process" and "process-file" edges, and the impossibility of mutual parent-child relationships between three process nodes, triangles in the provenance graph consist only of "process-process-file" combinations. This may imply the use of inter-process communication, such as pipeline communication between parent and child processes.

Therefore, given the scarcity of triangles in the provenance graph, the local clustering coefficient algorithm provides a unique and effective means of describing process behavior.

Regarding the cost of using topological features, the feature extraction algorithm used here only retrieves information from nodes within 2 hops. Therefore, even in the face of an expanding provenance graph in a production environment, each node change will only affect a small range of process nodes. This ensures that the cost of obtaining topological features on the provenance graph remains affordable.

The advantage of topological features lies in the ability to extract richer information from different subgraphs of

the provenance graph without incurring excessive computational burden. The use of topological features also extends the receptive field of the model to some extent, which is described in Sects. .

### Graph embedding

The graph embedding process of the ProcSAGE method is based on the GraphSAGE model (Hamilton et al. 2017). It trains the GraphSAGE model for process feature selection, learns the local information of the graph, and optimizes through clustering results, aiming to make embeddings of same labeled processes as close as possible and embeddings of different labeled processes as far apart as possible.

The GraphSAGE model (Hamilton et al. 2017) is a framework that uses the forward propagation algorithm to aggregate information from neighboring nodes. GraphSAGE takes graph information  $G = (V, E, X_v, X_e, T_e)$ , feature assignment functions  $F$ , number of hops  $K$ , neighborhood functions  $N : V \rightarrow 2^V$ , aggregator functions  $AGGREGATE_k$ , and nonlinear activation functions  $\sigma$  as inputs to train the weight matrices  $W^k, \forall k \in \{1, \dots, K\}$  in the GraphSAGE model. It outputs vector representations  $Z_v, \forall v \in V$  for each node.

The input graph for the GraphSAGE model is the process subgraph  $subG(procV, E, X_v, X_e, T_e)$  rather than the entire provenance graph, since the information about the file node is already captured in the topological features of the process nodes. This greatly reduces the computational overhead of the GraphSAGE model. In terms of other hyperparameters, the ProcSAGE method chooses the mean aggregator function and the ReLU activation function. The GraphSAGE model in the ProcSAGE method has  $K - 1$  hidden layers because aggregation occurs between two layers: the first aggregation occurs from the input layer to the hidden layer, and the last aggregation occurs from the hidden layer to the output layer.

During the training phase, ProcSAGE utilizes the Forward Propagation algorithm for multiple iterations. After each iteration, it calculates the clustering centers of nodes in the same class based on the clustering features and predicts the class membership of nodes. It then adjusts the weight matrices  $W^k$  based on the cross-entropy loss between the predicted results and the true labels. Through this process, the model explores different node representation weights through forward propagation and parameter tuning to learn the local structure adjacent to the nodes, aiming to find the optimal feature extraction scheme for clustering.

During the clustering process, ProcSAGE adopts the idea of K-means. For nodes with the same label, ProcSAGE calculates their geometric mean as the clustering center for that category. Then, when predicting the class membership of nodes, ProcSAGE assigns nodes to the class corresponding to the clustering center with the smallest Euclidean distance. Based on these predictions, the GraphSAGE model computes the cross-entropy of the predicted results and iteratively adjusts the model parameters to better extract process graph embeddings. The pseudo-code implementation can be seen in Algorithm 1.

The clustering model used in ProcSAGE is different from common clustering models. Typically, clustering models determine clustering centers and assign data points to clusters based on fixed feature embeddings. ProcSAGE takes a different approach. Instead of determining clustering centers, it modifies the clustering embeddings to obtain clusters. Specifically, by adjusting the parameters of the GraphSAGE model, embeddings of processes with the same label are made as close as possible in the feature space to accomplish the clustering goals.

#### Algorithm 1 Graph Embedding Model Training

---

**Input:** Provenance graph  $G = (V, E, X_v, X_e, T_e)$ , feature mapping function  $F$ , label mapping function  $L$ , hop number  $K$ , forward propagation (FP) training epochs *epoch*

**Output:** GraphSAGE model parameters  $W^k$ , clustering centers  $CC$ , and anomaly thresholds *thresholds*

```

1:  $procV \leftarrow$  process node
2:  $subG \leftarrow (procV, E, X_v, X_e, T_e)$ 
3: for  $i = 0$  to epoch do
4:    $Z \leftarrow FP(subG, F, K, AGGREGATE_k, W^k, \sigma)$ 
5:    $CC \leftarrow Calc\_Clustering\_Center(Z, L(procV))$ 
6:    $dis \leftarrow Dis\_to\_Center(Z, CC)$ 
7:    $pred\_label \leftarrow argmin(dis)$ 
8:    $loss \leftarrow cross\_entropy(pred\_label, L(procV))$ 
9:    $W^k \leftarrow backward(loss)$ 
10: end for
11:  $thresholds \leftarrow find\_proper\_thresholds(dis)$ 

```

---

### Anomaly detection

In the detection phase, the same method as in the training phase is used to extract node features from the input audit log stream. The pre-trained GraphSAGE model is then used to compute the corresponding clustering



embeddings and the Euclidean distance from the embeddings to each clustering center. Since no attacks are assumed to occur during the training phase, the trained clustering centers do not contain any malicious centers. The measure of whether a process node is anomalous is based on its outlier position.

ProcSAGE uses two methods to determine if a node is an outlier. The first method uses a fixed distance threshold. If the distance between a node and its nearest clustering center exceeds this threshold, the node is considered an outlier. The second method uses a relative distance threshold. It calculates the ratio of a node's distance to its second nearest clustering center to its distance to its nearest clustering center. If this ratio is below the threshold, indicating that the node is not particularly close to any clustering center, the node is considered an "outlier" and an alert is triggered.

### Method evaluation

This section discusses the effectiveness of ProcSAGE. The performance evaluation experiments are performed on a server running Ubuntu 22.04.1 LTS. The server was equipped with an Intel(R) Xeon(R) Silver 4210 CPU @ 2.20GHz, paired with a Tesla P100-PCIE-16GB graphics card, and possessed a memory capacity of 125 G. Multiple datasets from StreamSpot are used in this study to evaluate the proposed method. The graph neural network algorithm utilized in the experiments is implemented using the PyG (PyTorch Geometric) library.

### Dataset

This paper uses an public log dataset, StreamSpot, which describes a set of syscall information capturing browser behavior. The dataset creator, Sadegh, uses Selenium Remote Control to automate tasks on the Firefox browser and uses Linux SystemTap to collect syscall records during execution.

Based on the different tasks performed by the browser, the dataset can be divided into five benign scenarios and one attack scenario. The benign scenarios simulate regular browsing activities of ordinary users, including activities such as watching YouTube, downloading files, browsing cnn.com, checking Gmail, and playing video games. The attack scenario simulates a user accessing a malicious URL, where the attacker exploits a Flash vulnerability to gain root access to the user's host and initiate stealthy downloads. Each complete simulated operation in the dataset is called a graph, which can generate a provenance graph. There are 100 graphs involved in each scenario. Statistical data for each scenario is summarized in Table 1.

**Table 1** StreamSpot datasets overview

| Dataset    | Scenario | Avg.   E | Avg.   V | # of graphs |
|------------|----------|----------|----------|-------------|
| StreamSpot | GMail    | 37,382   | 6827     | 100         |
|            | Download | 310,814  | 8831     | 100         |
|            | CNN      | 294,903  | 8990     | 100         |
|            | VGame    | 112,958  | 8637     | 100         |
|            | YouTube  | 113,229  | 8292     | 100         |
|            | Attack   | 28,423   | 8891     | 100         |

**Table 2** Syscall groups used in StreamSpot preprocessing

| Group id | Syscalls                                                                            |
|----------|-------------------------------------------------------------------------------------|
| 1        | recv, recvfrom, recvmsg, send, sendto, sendmsg, bind, connect, listen, accept, open |
| 2        | waitpid, clone, fork, execl, execv, execle, execve, execlp, execvp                  |
| 3        | chmod, chown, unlink, truncate, ftruncate                                           |
| 4        | write, pwrite, writev, pwritev, read                                                |

### Data preprocessing

To obtain the semantic features from the StreamSpot dataset, the occurrences of syscalls (such as open, read, recv, etc.) are first counted, denoted as  $N_s$ . The syscall domain is defined as  $\mathbb{S}$ , including all syscalls that occur in the dataset. The mapping  $M : \mathbb{S} \rightarrow \mathbb{N}$  is used to represent the mapping of syscalls to natural numbers  $[0, N_s - 1]$ . For a relevant process in an audit log  $l$ , if the process  $p$  serves as an input for the information flow (control flow or data flow) during the execution of a syscall (for example, *exec* an executable file, *read* a data file, or being *cloned* by a parent process), it is defined as  $l \in In(p)$ ; if the process  $p$  serves as an output for the information flow (for example, *chmod* modifying file status, *send* a string to a network socket, or *clone* a child process), it is defined as  $l \in Out(p)$ .  $l.syscall$  represents the syscall names involved in the log. The function  $F^{semantic} : V \rightarrow \mathbb{N}^{2 \times N_s}$  that extracts the semantic feature for each process is defined as  $F^{semantic}_{(p)} = [a_0, a_1, \dots, a_{N_s \times 2 - 1}]$ , where:

$$a_i = \begin{cases} \#\{l \in In(p) \ \& \ M(l.syscall) == i\}, & i < N_s \\ \#\{l \in Out(p) \ \& \ M(l.syscall) == i - N_s\}, & i \geq N_s \end{cases}$$

To extract the topological features of processes on the provenance graph from the dataset, this paper developed a parser for the dataset to generate the provenance graph from the audit logs. In this study, the syscall groups in Table 2 were selected to improve the information content and interpretability of the topological features for the syscalls present in the StreamSpot dataset. The grouping

method is primarily based on the functional characteristics of the syscalls, grouping functionally similar syscalls together. For example, syscall group 1 includes syscalls related to sockets and network communication, reflecting the network-side characteristics of the process. Similarly, syscall groups 3 and 4 contain syscalls related to file management and file reading/writing, respectively, reflecting the file-side characteristics of the process. Based on these syscall groups, this paper preprocessed the provenance graph as described in Sect. , and extracted the average degree of neighbors and local clustering coefficient of processes from the provenance graph and syscall group subgraphs. These data serve as topological features  $F_{(p)}^{topological}$  of the processes.

In summary, the feature mapping function  $F_{(p)}$  in this data set is defined as  $F_{(p)} = F_{(p)}^{semantic} || F_{(p)}^{topological}$ .

Regarding the selection of clustering centers, since the StreamSpot dataset does not provide process names and file names, it is not possible to infer the behavior of processes based on their names. Therefore, processes are roughly grouped by scenario, with the goal of keeping processes within the same scenario as close together as possible.

### Experiment results and analyses

In this section, the paper has designed four experiments to validate the ProcSAGE method. First, we validate the accuracy of ProcSAGE and compare it with the other baseline models. Second, we demonstrate the effectiveness of the clustering model in ProcSAGE. Furthermore, we discuss the influence of different receptive fields on our model. Finally, we compare the time and memory consumption of the ProcSAGE method with the StreamSpot model across different data scales.

#### Accuracy validation experiment

This paper compares ProcSAGE with three baseline anomaly detection models: two state-of-the-art provenance graph methods StreamSpot (Manzoor et al. 2016a) and UNICORN (Han et al. 2020), and a graph neural network model Graph Attention Network (GAT)

**Table 3** Results of different methods on StreamSpot dataset

| Methods         | Precision | Recall | Accuracy | F1-score |
|-----------------|-----------|--------|----------|----------|
| StreamSpot      | 0.77      | 0.84   | 0.74     | 0.80     |
| UNICORN (R = 1) | 0.51      | 1.0    | 0.60     | 0.68     |
| GAT             | 0.86      | 0.91   | 0.88     | 0.88     |
| UNICORN (R = 3) | 0.98      | 0.93   | 0.96     | 0.94     |
| ProcSAGE        | 0.96      | 1.0    | 0.98     | 0.98     |

(Velickovic et al. 2017) that allocates different weights to different neighboring nodes using self-attention layers. Specifically, we further divide UNICORN into two cases based on the size of the receptive field,  $R = 1$  (small receptive field) and  $R = 3$  (large receptive field). During the comparison, 80% of benign data from the dataset is used as training data, and the remaining 20% along with malicious data is used as test data. The learning rate during training is set to 0.01, and the weight decay is  $5e - 4$ .

As shown in Table 3, this paper uses four metrics—precision, recall, accuracy, and F1-score—for a comprehensive comparison of different models. It can be observed that the proposed model ProcSAGE performs the best, achieving accuracy and F1-score of 98%. Compared to the StreamSpot model originally used with this dataset, the performance improvement exceeds 20%.

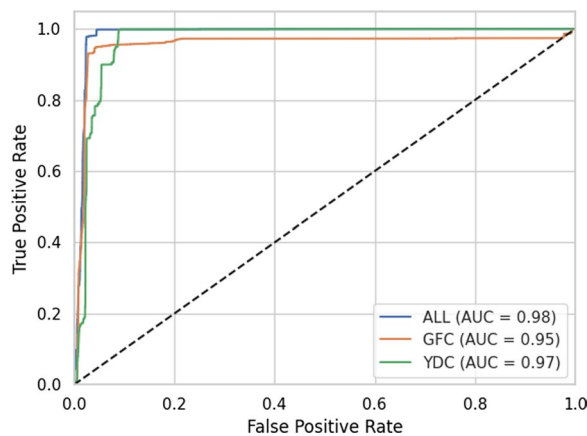
To thoroughly discuss model performance, this paper further divides the dataset into three sub-datasets, following the approach of the StreamSpot dataset authors. Since the dataset comprises 5 benign scenarios and 1 attack scenario, different sub-datasets include different benign scenarios but all share the same attack scenario. The specific benign scenarios included in each sub-dataset are outlined in Table 4.

**Table 4** Overview of the StreamSpot subdatasets (benign scenario only)

| Subdataset | Scenario                         | Avg.   E | Avg.   V | # of graphs |
|------------|----------------------------------|----------|----------|-------------|
| ALL        | GMail,download,CNN,VGame,YouTube | 173,857  | 8315     | 500         |
| GVC        | GMail,VGame,CNN                  | 148,414  | 8151     | 300         |
| YDC        | YouTube,download,CNN             | 239,648  | 8705     | 300         |

**Table 5** Result of different methods on StreamSpot SubDataset

| Methods         | SubDataset | Precision | Recall | Accuracy | F1-score |
|-----------------|------------|-----------|--------|----------|----------|
| StreamSpot      | YDC        | 0.73      | 0.70   | 0.73     | 0.72     |
|                 | GFC        | 0.71      | 0.76   | 0.72     | 0.74     |
|                 | ALL        | 0.77      | 0.84   | 0.74     | 0.80     |
| GAT             | YDC        | 0.82      | 0.84   | 0.74     | 0.83     |
|                 | GFC        | 0.83      | 0.91   | 0.79     | 0.87     |
|                 | ALL        | 0.86      | 0.91   | 0.88     | 0.88     |
| UNICORN (R = 3) | YDC        | 0.91      | 0.92   | 0.86     | 0.92     |
|                 | GFC        | 0.92      | 0.95   | 0.90     | 0.94     |
|                 | ALL        | 0.98      | 0.93   | 0.96     | 0.94     |
| ProcSAGE        | YDC        | 0.96      | 0.98   | 0.95     | 0.97     |
|                 | GFC        | 0.94      | 0.95   | 0.92     | 0.94     |
|                 | ALL        | 0.96      | 1.0    | 0.98     | 0.98     |



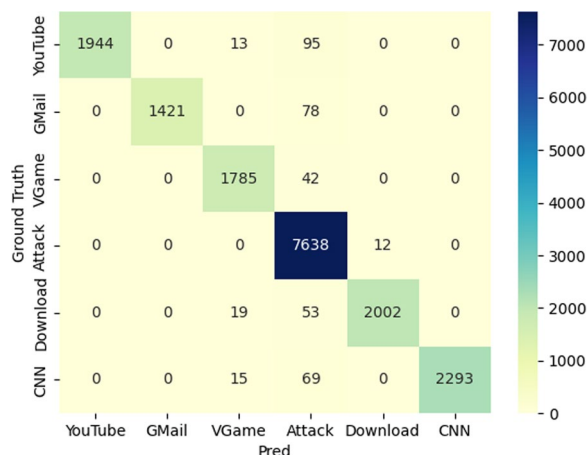
**Fig. 3** ROC curves of the ProcSAGE method on the subdatasets

The accuracy of the ProcSAGE method compared to other models on the three subdatasets is presented in Table 5. It can be seen that ProcSAGE has a good performance on data sets of different scales.

To visually demonstrate the performance of this model on three sub-datasets, ROC curves are plotted by adjusting the distance threshold, as shown in Fig. 3. It can be observed that the AUC is greater than or equal to 0.95 for all three datasets. This indicates that there is a significant distinction between benign and malicious processes under the ProcSAGE method, and that ProcSAGE can effectively discriminate between benign and malicious processes by adjusting the threshold.

#### Clustering validation experiment

Considering that the StreamSpot dataset is divided by scenarios, this paper's model predicts the scene to which a process belongs based on the process's proximity to the clustering center during the clustering process. The



**Fig. 4** Scenario Prediction Confusion Matrix

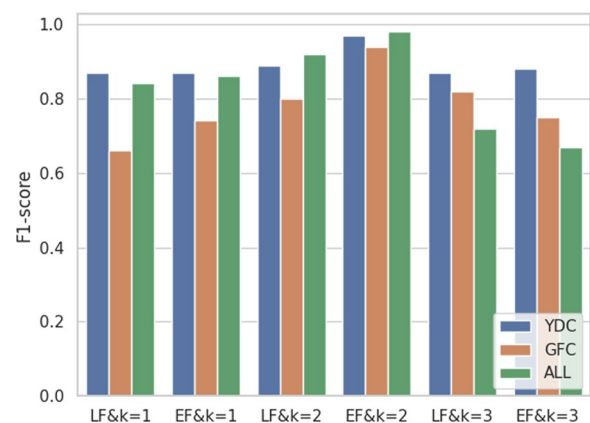
heatmap of the predicted result is illustrated in Fig. 4. It can be observed that the model possesses excellent characterization capabilities for processes. Not only can it effectively detect malicious processes in terms of anomaly detection, but it also exhibits the ability to learn and predict process features in different scenarios. Further analysis shows that predictions under benign scenarios are generally accurate, with most errors concentrated in anomaly detection, which determines whether a process belongs to an attack scenario. This is due to the lack of attack scenario data during the model's training phase, resulting in certain mispredictions.

#### Receptive field comparison experiment

To further illustrate the characteristics of ProcSAGE, Fig. 5 compares the F1-scores of the model with different receptive field sizes on three subdatasets.

ProcSAGE characterizes processes based on their neighbor nodes in the provenance graph. Therefore, the size of the receptive field used in the model directly affects the characterization of processes. In the model, two factors influence the receptive field: one is the use of topological features extracted from the provenance graph, which expands the receptive field, and the other is the number of hops in the GraphSAGE model. Therefore, in the comparative experiment, this paper controls variables on the feature side, choosing to use only the limited features (LF) of the process semantic features, or extended features (EF) with both the semantic features and topological features. On the GraphSAGE model side, the number of hops is controlled at  $k = 1, 2$ , or  $3$ .

From the results in Fig. 5, it can be observed that when the receptive field is small, the model's characterization ability for processes is weak. As the receptive field expands, the model's performance gradually improves. However, when the receptive field is too large, it leads



**Fig. 5** F1-scores for Different Receptive fields

to the dilution of the process's own features, resulting in feature convergence for all processes and a decrease in clustering effectiveness.

### Performance experiment

This paper designs a performance comparison experiment to compare the time and memory cost of the ProcSAGE method with other models on datasets of different scales.

In this experiment, we truncate the StreamSpot dataset. For this experiment, 20%, 40%, 60%, 80%, and all subgraphs are extracted as datasets while maintaining the ratio of the number of subgraphs for each scenario. Since the subgraphs in the StreamSpot dataset are independent of each other, and the graph size of subgraphs under the same scenario is similar, this truncation method ensures a linear growth relationship in graph size for the five data subsets.

The experiment used the same ProcSAGE parameters as in the accuracy validation experiment. For the Streamspot models, default parameter configurations recommended in the paper or code were utilized. Due to the unsatisfactory performance of the GAT model, the parameter *epoch* = 300 is set for the GAT model for comparison purposes, even though its accuracy may not be optimal under these conditions.

The paper used the Linux *time* command to record time overhead. For Python code, the maximum memory overhead was recorded using the *memory\_profiler* package. For C++ code, a script is used to record the process memory usage every second during execution in order to get the maximum memory overhead. Since the StreamSpot model does not use the GPU, all models were processed on the CPU for fairness.

The experimental results are shown in Fig. 6. It can be observed that, compared to models like StreamSpot, which use complete provenance graph representation

learning, ProcSAGE reduces time consumption by 69% and memory consumption by 78% when processing the StreamSpot dataset. This is because ProcSAGE focuses on and computes process and thread nodes when determining host anomalies, rather than computing all nodes in the provenance graph.

The provenance graph reflects the behavioral relationships of processes in the system, but process and thread nodes represent only a small portion of the node set in the provenance graph. In contrast, the majority of the remaining nodes are configuration files, data files, cache files, and network socket nodes that processes interact with at runtime. The ProcSAGE method aggregates information from these nodes into process and thread nodes, avoiding separate computations for these nodes.

As the size of the dataset and the number of files and socket nodes that processes interact with increases, the effectiveness of this performance optimization becomes more apparent. Figure 6 also shows that the growth rate of ProcSAGE consumption is much smaller than the StreamSpot model as the dataset scale and provenance graph size increase.

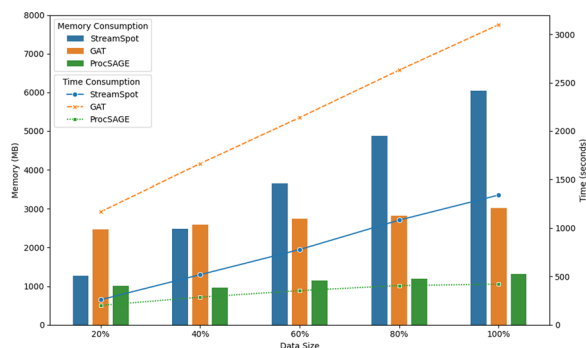
### Future work

Despite the ProcSAGE method having achieved significant progress in advanced threat detection and performance optimization, there are still some limitations and unresolved problems. In these concerns, we proposes several directions for future research:

**Streaming processing capability** For advanced threat detection models, the continuous monitoring of host anomaly states by streaming the system call audit logs is a valuable research direction. While the ProcSAGE method has fully considered this requirement in its design, minimizing the computational burden of newly generated data to enable streaming processing, future research can dive deeper into this direction.

**Model transferability** The ProcSAGE method builds the model during the training phase and does not adjust it during the detection phase, potentially leading to concept drift issues over long timescale detection periods. Future work may explore methods to allow the model to adapt and adjust itself over time.

**Learning in complex environments** Hosts acting as servers may exhibit certain regularities in behavior, while hosts such as personal computers may exhibit more complicated behavior. Effectively detecting advanced threats within the complex behavior of hosts proposes a common challenge for anomaly models. This complexity presents a critical area for future research.



**Fig. 6** Consumption at different data scales



## Conclusion

In this study, we introduced the ProcSAGE method, a novel approach designed to address the significant challenge of detecting APTs in massive system audit logs. By focusing on processes or threads and considering both their attributes and those of their neighboring nodes, ProcSAGE effectively reduces the scale and complexity of provenance graphs, significantly lowering the computational resources required for graph construction and feature extraction.

The benchmarking on the StreamSpot dataset validates the efficacy of ProcSAGE, demonstrating its capability to substantially decrease time and memory requirements while concurrently enhancing detection accuracy. Notably, the benefits of ProcSAGE increase as the volume of data increases, highlighting its scalability and practicality in real-world scenarios where audit logs grow exponentially.

## Acknowledgements

We thank all reviewers for their comments and suggestions.

## Author contributions

BX designed the proposed method, set up the experiment, and wrote the draft. YG, XG and YL optimized the design of the method. CD, SL, YL, BJ and ZL revised the manuscript. All authors read and approved the final manuscript.

## Funding

This research is supported by National Key Research and Development Program of China (No. 2023YFC2206402), Youth Innovation Promotion Association CAS (No. 2021156), the Strategic Priority Research Program of the Chinese Academy of Sciences (No. XDC02040100), Foundation Strengthening Program Technical Area Fund, 021-JCJQ-JJ-0908, and State Grid Corporation of China Science and Technology Program (Contract No.: SG270000YXJS2311060). This work is also supported by the Program of Key Laboratory of Network Assessment Technology, the Chinese Academy of Sciences, Program of Beijing Key Laboratory of Network Security and Protection Technology.

## Availability of data and materials

The StreamSpot dataset is available from <https://github.com/sbustreamspot/sbustreamspot-data>

## Declarations

## Competing interests

The authors declare that they have no Conflict of interest.

Received: 21 January 2024 Accepted: 6 April 2024

Published online: 25 August 2024

## References

- Alsaheel A, Nan Y, Ma S, Yu L, Walkup G, Celik ZB, Zhang X, Xu D (2021) A sequence-based learning approach for attack investigation. In: Proceedings of the 30th security symposium
- Alshamrani A, Myneni S, Chowdhary A, Huang D (2019) A survey on advanced persistent threats: techniques, solutions, challenges, and research opportunities. *IEEE Commun Surv Tutor* 21(2):1851–1877
- Altinisik E, Deniz F, Sencar HT (2023) Provg-searcher: a graph representation learning approach for efficient provenance graph search. In: Proceedings of the 2023 ACM SIGSAC conference on computer and communications security, pp 2247–2261
- Binde B, McRee R, O'Connor TJ (2011) Assessing outbound traffic to uncover advanced persistent threat. SANS Institute. Whitepaper 16
- Cheng Z, Lv Q, Liang J, Wang Y, Sun D, Pasquier T, Han X (2023) Kairos: practical intrusion detection and investigation using whole-system provenance. *arXiv preprint arXiv:2308.05034*
- Fang P, Gao P, Liu C, Ayday E, Jee K, Wang T, Ye YF, Liu Z, Xiao X (2022) {Back-Propagating} system dependency impact for attack investigation. In: 31st USENIX security symposium (USENIX Security 22), pp 2461–2478
- Fei P, Li Z, Wang Z, Yu X, Li D, Jee K (2021) {SEAL}: Storage-efficient causality analysis on enterprise logs with query-friendly compression. In: 30th USENIX security symposium (USENIX Security 21), pp 2987–3004
- Forrest S, Hofmeyr SA, Somayaji A, Longstaff TA (1996) A sense of self for unix processes. In: Proceedings 1996 IEEE symposium on security and privacy, pp 120–128. IEEE
- Hamilton W, Ying Z, Leskovec J (2017) Inductive representation learning on large graphs. *Adv Neural Inf Process Syst* 30
- Han X, Pasquier TFJ, Bates A, Mickens J, Seltzer MI (2020) Unicorn: runtime provenance-based detector for advanced persistent threats. In: 27th Annual network and distributed system security symposium, NDSS 2020, San Diego, California, USA, February 23–26, 2020. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/unicorn-runtime-provenance-based-detector-for-advanced-persistent-threats/>
- Hassan WU, Bates A, Marino D (2020) Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE symposium on security and privacy (SP), pp 1172–1189. IEEE
- Hassan WU, Guo S, Li D, Chen Z, Jee K, Li Z, Bates A (2019) Nodooze: combatting threat alert fatigue with automated provenance triage. In: Network and distributed systems security symposium
- Hassan WU, Li D, Jee K, Yu X, Zou K, Wang D, Chen Z, Li Z, Rhee J, Gui J, et al (2020) This is why we can't cache nice things: lightning-fast threat hunting using suspicion-based hierarchical storage. In: Annual computer security applications conference, pp 165–178
- Hossain MN, Milajerdi SM, Wang J, Eshete B, Gjomemo R, Sekar R, Stoller S, Venkatakrishnan V (2017) {SLEUTH}: real-time attack scenario reconstruction from {COTS} audit data. In: 26th USENIX security symposium (USENIX Security 17), pp 487–504
- Hossain MN, Sheikh S, Sekar R (2020) Combating dependence explosion in forensic analysis using alternative tag propagation semantics. In: 2020 IEEE symposium on security and privacy (SP), pp 1139–1155. IEEE
- Hossain MN, Wang J, Weisse O, Sekar R, Genkin D, He B, Stoller SD, Fang G, Piessens F, Downing E, et al (2018) {Dependence-Preserving} data compaction for scalable forensic analysis. In: 27th USENIX security symposium (USENIX Security 18), pp 1723–1740
- King ST, Chen PM (2003) Backtracking intrusions. In: Proceedings of the nineteenth ACM symposium on operating systems principles, pp 223–236
- Kruegel C, Kirda E, Mutz D, Robertson W, Vigna G (2005) Automating mimicry attacks using static binary analysis. In: USENIX security symposium, vol 14, pp 11–11
- Lee KH, Zhang X, Xu D (2013) Loggc: garbage collecting audit log. In: Proceedings of the 2013 ACM SIGSAC conference on computer & communications security, pp 1005–1016
- Li Z, Cheng X, Sun L, Zhang J, Chen B (2021) A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks. *Secur Commun Netw* 2021:1–14
- Liu F, Wen Y, Zhang D, Jiang X, Xing X, Meng D (2019) Log2vec: a heterogeneous graph embedding based approach for detecting cyber threats within enterprise. In: Proceedings of the 2019 ACM SIGSAC conference on computer and communications security, pp 1777–1794
- Liu Y, Zhang M, Li D, Jee K, Li Z, Wu Z, Rhee J, Mittal P (2018) Towards a timely causality analysis for enterprise security. In: NDSS
- Manzoor E, Milajerdi SM, Akoglu L (2016a) Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1035–1044
- Manzoor E, Milajerdi SM, Akoglu L (2016b) Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In: Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, pp 1035–1044
- Milajerdi SM, Eshete B, Gjomemo R, Venkatakrishnan V (2019b) Poirot: aligning attack behavior with kernel audit records for cyber threat hunting. In:

- Proceedings of the 2019 ACM SIGSAC conference on computer and communications security, pp. 1795–1812
- Milajerdi SM, Gjormemo R, Eshete B, Sekar R, Venkatakrishnan V (2019a) Holmes: real-time apt detection through correlation of suspicious information flows. In: 2019 IEEE symposium on security and privacy (SP), pp 1137–1152. IEEE
- Pasquier T, Han X, Goldstein M, Moyer T, Eysers D, Seltzer M, Bacon J (2017) Practical whole-system provenance capture. In: Proceedings of the 2017 symposium on cloud computing, pp 405–418
- Pei K, Gu Z, Saltaformaggio B, Ma S, Wang F, Zhang Z, Si L, Zhang X, Xu D (2016) Hercule: attack story reconstruction via community discovery on correlated log graph. In: Proceedings of the 32Nd annual conference on computer security applications, pp 583–595
- Sun X, Dai J, Liu P, Singhal A, Yen J (2018) Using bayesian networks for probabilistic identification of zero-day attack paths. *IEEE Trans Inf Forensics Secur* 13(10):2506–2521
- Tan KM, Maxion RA (2002) “ why 6?” defining the operational limits of stide, an anomaly-based intrusion detector. In: Proceedings 2002 IEEE symposium on security and privacy, pp 188–201. IEEE
- Velickovic P, Cucurull G, Casanova A, Romero A, Lio P, Bengio Y et al (2017) Graph attention networks. *Stat* 1050(20):10–48550
- Wagner D, Soto P (2002) Mimicry attacks on host-based intrusion detection systems. In: Proceedings of the 9th ACM conference on computer and communications security, pp 255–264
- Wang S, Wang Z, Zhou T, Sun H, Yin X, Han D, Zhang H, Shi X, Yang J (2022) Threatrace: detecting and tracing host-based threats in node level through provenance graph learning. *IEEE Trans Inf Forensics Secur* 17:3972–3987
- Wang Q, Hassan WU, Li D, Jee K, Yu X, Zou K, Rhee J, Chen Z, Cheng W, Gunter CA, et al (2020) You are what you do: Hunting stealthy malware via data provenance analysis. In: NDSS
- Warrender C, Forrest S, Pearlmutter B (1999) Detecting intrusions using system calls: alternative data models. In: Proceedings of the 1999 IEEE symposium on security and privacy (Cat. No. 99CB36344), pp 133–145. IEEE
- Xiong C, Zhu T, Dong W, Ruan L, Yang R, Cheng Y, Chen Y, Cheng S, Chen X (2020) Conan: a practical real-time apt detection system with high accuracy and efficiency. *IEEE Trans Dependable Secure Comput* 19(1):551–565
- Xu Z, Fang P, Liu C, Xiao X, Wen Y, Meng D (2022) Depcomm: Graph summarization on system audit logs for attack investigation. In: 2022 IEEE symposium on security and privacy (SP), pp 540–557. IEEE
- Xu Z, Wu Z, Li Z, Jee K, Rhee J, Xiao X, Xu F, Wang H, Jiang G (2016) High fidelity data reduction for big data security dependency analyses. In: Proceedings of the 2016 ACM SIGSAC conference on computer and communications security, pp 504–516
- Zengy J, Wang X, Liu J, Chen Y, Liang Z, Chua T-S, Chua ZL (2022) Shade-watcher: Recommendation-guided cyber threat analysis using system audit records. In: 2022 IEEE symposium on security and privacy (SP), pp 489–506. IEEE
- Zhu T, Wang J, Ruan L, Xiong C, Yu J, Li Y, Chen Y, Lv M, Chen T (2021) General, efficient, and real-time data compaction strategy for apt forensic analysis. *IEEE Trans Inf Forensics Secur* 16:3312–3325
- Zimba A, Chen H, Wang Z, Chishimba M (2020) Modeling and detection of the multi-stages of advanced persistent threats attacks based on semi-supervised learning and complex networks characteristics. *Future Gen Comput Syst* 106:501–517

## Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.